

CẨM NANG HÀM PYTHON NỀN TẢNG CHO AI & MACHINE LEARNING

Tổng hợp cú pháp & ví dụ ứng dụng thực tế từ các thư viện `os`, `random`, và `math`

Trong quá trình học và làm việc với Trí tuệ Nhân tạo (AI), Machine Learning (ML) và Deep Learning, việc làm chủ các thư viện tiêu chuẩn của Python như **os** (quản lý tệp & thư mục dữ liệu), **random** (khởi tạo ngẫu nhiên & xáo trộn dữ liệu) và **math** (tính toán toán học cơ bản) là vô cùng quan trọng. Dưới đây là bảng tổng hợp chi tiết dành riêng cho sinh viên và lập trình viên AI.

1. THƯ VIỆN OS — QUẢN LÝ HỆ THỐNG VÀ DỮ LIỆU

Thư viện `os` giúp tương tác với hệ điều hành, xử lý đường dẫn dataset, tạo thư mục lưu model checkpoint và đọc biến môi trường.

- **`os.path.join(path, *paths)`**

Mô tả & Ý nghĩa trong AI: Nối các thành phần đường dẫn một cách an toàn trên mọi hệ điều hành (Windows, Linux, macOS). Rất cần thiết khi trở tới thư mục dataset.

Cú pháp (Syntax): `os.path.join('folder', 'subfolder', 'filename.csv')`

```
import os

data_dir = 'dataset/images'
file_name = 'cat_01.jpg'
full_path = os.path.join(data_dir, file_name)
print('Đường dẫn: ', full_path)
```

- **`os.makedirs(name, exist_ok=True)`**

Mô tả & Ý nghĩa trong AI: Tạo thư mục đa cấp (tạo cả các thư mục cha nếu chưa tồn tại). Tham số `exist_ok=True` ngăn lỗi nếu thư mục đã tồn tại, rất hay dùng khi lưu model checkpoints.

Cú pháp (Syntax): `os.makedirs('checkpoints/resnet50/v1', exist_ok=True)`

```
import os

checkpoint_dir = 'models/checkpoints/run_01'
os.makedirs(checkpoint_dir, exist_ok=True)
print(f'Đã sẵn sàng thư mục lưu model: {checkpoint_dir}')
```

- **`os.listdir(path='.')` & **`os.walk(top)`****

Mô tả & Ý nghĩa trong AI: `os.listdir` liệt kê tất cả tệp/thư mục trong một đường dẫn. `os.walk` duyệt cây thư mục đệ quy, dùng để quét toàn bộ dataset ảnh/bản tin.

Cú pháp (Syntax): `os.listdir('dataset/')` | `os.walk('data_root/')`

```
import os

data_root = 'data/train'
if os.path.exists(data_root):
    for root, dirs, files in os.walk(data_root):
        print(f'Thư mục: {root} có {len(files)} file')
```

- **os.environ**

Mô tả & Ý nghĩa trong AI: Mapping quản lý các biến môi trường hệ thống. Trong AI, thường dùng để cấu hình GPU (CUDA), thiết lập API Key hoặc các tham số môi trường.

Cú pháp (Syntax): `os.environ['CUDA_VISIBLE_DEVICES'] = '0'`

```
import os

os.environ['CUDA_VISIBLE_DEVICES'] = '0'
os.environ['TOKENIZERS_PARALLELISM'] = 'false'
print('CUDA GPU configured:', os.getenv('CUDA_VISIBLE_DEVICES'))
```

2. THƯ VIỆN RANDOM — KHỞI TẠO NGẪU NHIÊN VÀ XÁO TRỘN DỮ LIỆU

Khởi tạo ngẫu nhiên đóng vai trò cốt lõi trong việc chia tập dữ liệu (train/val/test split), xáo trộn mẫu (shuffling) và thiết lập Seed để tái lập kết quả thử nghiệm.

- **random.seed(a)**

Mô tả & Ý nghĩa trong AI: Thiết lập sinh số ngẫu nhiên cố định. Trong AI, việc cố định seed giúp tái lập kết quả thí nghiệm (Reproducibility), đảm bảo chia dữ liệu hay khởi tạo trọng số luôn nhất quán.

Cú pháp (Syntax): `random.seed(42)`

```
import random

random.seed(42)
print('Số ngẫu nhiên 1:', random.random())
random.seed(42)
print('Số ngẫu nhiên 2 (giống kết quả 1):', random.random())
```

- **random.shuffle(x)**

Mô tả & Ý nghĩa trong AI: Xáo trộn trực tiếp danh sách (in-place). Thường dùng xáo trộn chỉ số (indices) hoặc danh sách đường dẫn tệp trước khi đưa vào Mini-batch.

Cú pháp (Syntax): `random.shuffle(data_list)`

```
import random

image_files = ['img1.jpg', 'img2.jpg', 'img3.jpg', 'img4.jpg']
random.shuffle(image_files)
print('Danh sách ảnh sau xáo trộn:', image_files)
```

- **random.sample(population, k)**

Mô tả & Ý nghĩa trong AI: Trích xuất k phần tử ngẫu nhiên không hoàn lại từ danh sách. Thường dùng để lấy mẫu nhanh (Subsampling) kiểm tra thử mô hình hoặc chia tập Validation.

Cú pháp (Syntax): `random.sample(dataset, k=100)`

```
import random

dataset = [f'sample_{i}' for i in range(1000)]
val_set = random.sample(dataset, 100)
print('Số mẫu validation:', len(val_set))
```

- **random.gauss(mu, sigma) & random.uniform(a, b)**

Mô tả & Ý nghĩa trong AI: Sinh số ngẫu nhiên phân phối Chuẩn (Gaussian) hoặc phân phối Đều (Uniform). Dùng trong thêm nhiễu (Noise Injection), Data Augmentation hoặc khởi tạo tham số.

Cú pháp (Syntax): random.gauss(mu=0.0, sigma=1.0) | random.uniform(a=-1.0, b=1.0)

```
import random

noise = random.gauss(0, 0.1)
weight = random.uniform(-0.5, 0.5)
print(f'Noise: {noise:.4f}, Weight: {weight:.4f}')
```

3. THƯ VIỆN MATH — TÍNH TOÁN TOÁN HỌC CƠ BẢN

Thư viện math cung cấp các hàm toán học C-level hiệu năng cao phục vụ tính toán Loss function, Activation function (Softmax, GELU, Sigmoid) và khoảng cách.

- **math.exp(x) & math.log(x, base)**

Mô tả & Ý nghĩa trong AI: Tính hàm mũ e^x và Logarithm tự nhiên. Đây là hai hàm cốt lõi để tính toán Activation Function (Sigmoid, Softmax) và hàm mất mát Cross-Entropy Loss.

Cú pháp (Syntax): math.exp(x) | math.log(x) # log tự nhiên base e

```
import math

def sigmoid(x):
    return 1 / (1 + math.exp(-x))

def cross_entropy_loss(y_true, y_pred):
    return - (y_true * math.log(y_pred) + (1 - y_true) * math.log(1 - y_pred))

print('Sigmoid(0):', sigmoid(0))
print('Loss (true=1, pred=0.9):', cross_entropy_loss(1, 0.9))
```

- **math.sqrt(x) & math.dist(p, q)**

Mô tả & Ý nghĩa trong AI: math.sqrt tính căn bậc hai. math.dist tính Khoảng cách Euclid (Euclidean distance) giữa 2 điểm, rất quan trọng trong thuật toán KNN, Clustering (K-Means).

Cú pháp (Syntax): math.sqrt(x) | math.dist([x1, y1], [x2, y2])

```
import math

point_a = [1.0, 2.0, 5.0]
point_b = [4.0, 6.0, 5.0]
euclidean_distance = math.dist(point_a, point_b)
print('Khoảng cách Euclid:', euclidean_distance)
```

- **math.erf(x)**

Mô tả & Ý nghĩa trong AI: Hàm Lỗi (Error Function). Được sử dụng trực tiếp trong công thức tính hàm kích hoạt GELU (Gaussian Error Linear Unit) phổ biến ở các mô hình Transformer (BERT, GPT, ViT).

Cú pháp (Syntax): math.erf(x)

```
import math

def gelu(x):
    return 0.5 * x * (1 + math.erf(x / math.sqrt(2)))

print('GELU(1.0):', gelu(1.0))
print('GELU(-1.0):', gelu(-1.0))
```

- **math.isclose(a, b, rel_tol=1e-9, abs_tol=0.0)**

Mô tả & Ý nghĩa trong AI: So sánh hai số thực (float) có gần bằng nhau hay không. Giúp tránh lỗi sai số làm tròn số thực khi kiểm tra Gradient (Gradient Checking) hoặc Unit Test trong AI.

Cú pháp (Syntax): `math.isclose(a, b, rel_tol=1e-5)`

```
import math

val1 = 0.1 + 0.2
val2 = 0.3
print('So sánh bằng math.isclose:', math.isclose(val1, val2))
```